

Ejercicios de Apache Kafka

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



CC BY-SA 4.0

Atribución-CompartirIgual

Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.

TEMA 2 – Apache Kafka

Se proponen diferentes ejercicios de ingesta y procesamiento de datos en streaming aplicados a un escenario de analítica deportiva. Para ello se utilizará **Apache Kafka** como sistema de mensajería distribuida. Se desarrollará un producer que publique mediciones de telemetría deportiva en un topic y distintos consumers que las consuman para **monitorizar condiciones de rendimiento y fisiología en tiempo real**.

En particular, se trabajará con un conjunto de datos de telemetría recogidos a través de sensores durante **3 partidos de fútbol para 15 jugadores**. Las mediciones incluyen parámetros de posición, velocidad, aceleración y frecuencia cardíaca. La generación en tiempo real de las mediciones se simulará a partir del script de Python *producer.py* y el archivo CSV *sensores_deportivos_fatiga.csv* proporcionados.

Ejercicio 1. Publicación de eventos con Kafka

Se pide implementar un **producer** que publique eventos con estas mediciones de telemetría en un topic de Kafka.

Apartado 1.1. Diseño del topic y del esquema del mensaje

Define el nombre del topic donde se publicarán los datos en streaming. Se recomienda seguir una convención de nombres jerárquica y con versionado de esquema.

A continuación, diseña el esquema de mensajes que se enviarán al topic. Debe contener los siguientes elementos:

- **Key:** debe elegirse una clave que permita preservar el orden de los eventos dentro de las particiones de un topic.
- **Value:** debe incluir todos los campos de medición y otros metadatos del mensaje relevantes. Se recomienda utilizar JSON e incorporar la versión del esquema de mensajes y un identificador único del evento.
- **Timestamp:** debe utilizarse el tiempo de evento asociado a cada medición y no el instante en el que se envía el mensaje.

Apartado 1.2. Implementación del producer

Modifica el script *producer.py* para que publique eventos en el topic de Kafka. Para ello, implementa las modificaciones necesarias en las líneas indicadas en el archivo. El identificador del evento puede generarse a partir de un [uuid](#).

Ejercicio 2. Consumo de eventos con Kafka

Se pide implementar distintos consumers de Kafka que se suscriban al topic definido en el ejercicio anterior y procesen en tiempo real los eventos publicados.

Apartado 2.1. Lectura de eventos

Implementa un consumer en Python que se suscriba al topic definido y lea los mensajes de forma continua. Para ello, se debe deserializar el contenido JSON de los mensajes enviados a los topics y mostrar por pantalla cada evento recibido.

Apartado 2.2. Alertas de frecuencia cardiaca

Implementa un segundo consumer que se suscriba al mismo topic e imprima únicamente por pantalla aquellos eventos que superen el umbral de frecuencia cardiaca de 180 bpm con una velocidad menor a 10 km/h.

Entregables

Se deberá entregar un único archivo comprimido en formato .zip en el Aula Virtual con los siguientes archivos Python:

- **producer.py**: Script del producer modificado para publicar eventos en el topic definido.
- **consumer_print.py**: Consumer que se suscribe al topic y muestra por pantalla cada evento recibido.
- **consumer_alert.py**: Consumer que se suscribe al topic e imprime únicamente los eventos que superen o igualen el umbral de frecuencia cardiaca configurado.

Debe incluirse una **memoria explicativa** en la que se describa el diseño del topic y el esquema de los mensajes empleados. Se deben especificar los requisitos del entorno y una guía clara de instalación y ejecución, indicando cómo lanzar el producer y los consumers.

MATERIAL DOCENTE EN ABIERTO

Ejercicios de Spark Structured Streaming

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



CC BY-SA 4.0

Atribución-CompartirIgual

Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y materiales
de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo
indicación expresa.

TEMA 2 – Spark Structured Streaming

Se proponen diferentes ejercicios de ingesta y procesamiento de datos en streaming aplicados a un escenario de analítica deportiva. Para ello se utilizará **Apache Spark Structured Streaming** para consumir y procesar en tiempo real los **eventos publicados** previamente en **Apache Kafka**. Se trabajará sobre el topic definido en la entrega anterior, asumiendo que el producer de Kafka ya está implementado. De este modo, el flujo de datos se genera en tiempo real a partir del **producer implementado en la entrega anterior**.

Para el desarrollo de la práctica se proporciona un entorno con Kafka y un Jupyter Notebook con PySpark mediante el fichero *docker-compose.yml*, así como un notebook base *spark_streaming.ipynb* con celdas marcadas para completar. El entorno levanta los servicios en local y monta el directorio de trabajo en el contenedor de Jupyter. Para simplificar la conectividad entre servicios, los contenedores comparten la red de la máquina anfitriona.

Ejercicio 1. Ingesta de eventos desde Kafka

Se pide implementar en PySpark un **stream de lectura desde Kafka** que consuma los eventos publicados en la práctica anterior.

La creación de la sesión de Spark necesaria para trabajar con Structured Streaming y el conector de Kafka ya se proporcionan dentro del notebook adjunto.

Apartado 1.1. Definición del esquema del mensaje

Define el esquema de Spark correspondiente al contenido de los mensajes publicados en Kafka. El **esquema debe incluir todos los campos** de metadatos y de medición de los eventos consumidos.

Apartado 1.2. Lectura del stream de eventos

Completa la constante *KAFKA_TOPIC* con el **nombre del topic definido** en la entrega anterior de Apache Kafka. Se debe reutilizar exactamente el mismo topic para consumir los eventos ya publicados.

Ejercicio 2. Procesamiento en streaming

Se pide transformar los mensajes leídos desde Kafka y **calcular métricas agregadas por jugador** mediante ventanas temporales en streaming. Se requiere usar Spark Structured Streaming.

Apartado 2.1. Parseo del mensaje

Implementa la transformación del stream para deserializar el contenido de los mensajes recibidos desde Kafka utilizando el esquema definido en el ejercicio anterior.

Debe tenerse en cuenta que el contenido de los mensajes en Kafka se recibe en bytes, por lo que será necesario **convertir su contenido a texto antes de aplicar el parseo del mensaje**. Para ello, puede utilizarse una transformación como `selectExpr('CAST(value AS STRING) AS json_str')`.

Apartado 2.2. Agregaciones por jugador en ventanas temporales

Implementa un flujo de agregación en streaming por jugador utilizando ventanas temporales deslizantes y watermark. La agregación debe agrupar, como mínimo, por **ventana temporal, partido y jugador**.

Se debe usar un **watermark de 2 minutos** para tolerar la llegada de eventos tardíos, así como **ventanas deslizantes de 1 minuto con deslizamiento de 30 segundos**.

Tras ello, se deben calcular las siguientes métricas por grupo, redondeando a dos decimales las métricas necesarias:

- Número de eventos
- Frecuencia cardiaca media
- Desviación estándar de la frecuencia cardiaca
- Frecuencia cardiaca máxima

Apartado 2.3. Salida del stream por consola

Completa la escritura del stream para mostrar los resultados en streaming utilizando el **modo update**. Para esta práctica no es necesario configurar checkpointing, ya que la salida se debe realizar **por consola** con fines de depuración y validación. La salida deberá permanecer activa mientras se estén publicando eventos desde Kafka.

Para facilitar la visualización de los resultados completos en consola, se recomienda desactivar el truncado de columnas mediante `option('truncate', 'false')` y consultar los logs del contenedor de Jupyter con el comando: `docker compose logs -f jupyter`.

Entregables

Se deberá entregar un único archivo comprimido en formato .zip en el Aula Virtual con los siguientes archivos:

- **spark_streaming.ipynb**: Notebook completado con todas las celdas / TODOS resueltas.

Debe incluirse una **memoria explicativa** en la que se describan el esquema del mensaje utilizado para deserializar el JSON y las transformaciones aplicadas sobre el stream. Asimismo, se deberá explicar la consulta en Spark Structured Streaming implementada. Se deben especificar los requisitos del entorno y una guía clara de instalación y ejecución. Esta memoria podrá entregarse como un documento único junto con la práctica anterior de Kafka, o bien como una memoria independiente para esta continuación.

MATERIAL DOCENTE EN ABIERTO

Ejercicios de Prompt Engineering y Post-training

Big Data Processing II

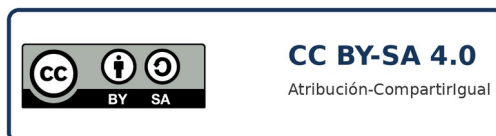
Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y
materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la
licencia, salvo indicación expresa.

Prompt Engineering aplicado al análisis deportivo

Objetivos

- Diseñar prompts profesionales y robustos.
- Controlar el comportamiento del modelo.
- Evitar alucinaciones.
- Generar salidas estructuradas reutilizables.

Ejercicio 1 — De prompt amateur a prompt profesional

Prompt original: «Analiza el rendimiento del jugador.»

- Identifique al menos 4 problemas del prompt.
- Rediseñe un prompt profesional incluyendo rol, contexto, restricciones y formato de salida.
- Explique por qué su versión mejora la fiabilidad.

Ejercicio 2 — Control de alucinaciones con datos incompletos

Datos: minutes=410, goals=2, assists=1, xG=1.3, shots_p90=2.1

- Diseñe un prompt que prohíba inventar datos e indique cómo actuar si faltan métricas.
- Solicite una salida estructurada.
- Explique cómo su prompt reduce el riesgo de alucinación.

Ejercicio 3 — Diseño de salida estructurada

- Diseñe un prompt que genere JSON con resumen técnico, fortalezas, debilidades, alertas y métricas utilizadas.
- Explique por qué este formato es preferible al texto libre.

Ejercicio 4 — Zero-shot vs Few-shot

Evento: Recuperación en campo propio, pase largo y finalización en 7 segundos.

- Diseñe un prompt zero-shot para clasificar el evento.
- Diseñe un prompt few-shot con ejemplos.
- Compare consistencia, claridad y formato.

Ejercicio 5 — Chain-of-Thought en análisis deportivo

Pregunta: ¿Por qué ha disminuido el rendimiento ofensivo?

- Diseñe un prompt normal.
- Diseñe un prompt que fuerce razonamiento paso a paso.
- Compare claridad, utilidad y riesgos.

Rúbrica de evaluación

- Excelente (9-10): Prompt completo, preciso, estructurado y bien justificado.
- Notable (7-8): Prompt correcto con pequeñas mejoras posibles.
- Aprobado (5-6): Cumple requisitos mínimos pero con ambigüedades.
- Insuficiente (<5): Prompt incompleto, ambiguo o sin justificación.

MATERIAL DOCENTE EN ABIERTO

Ejercicios de Retrieval-Augmented Generation

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



CC BY-SA 4.0

Atribución-CompartirIgual

Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y
materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la
licencia, salvo indicación expresa.

RAG aplicado al análisis deportivo

Se plantean diferentes ejercicios basados en técnicas simples de RAG.

Objetivo: comprender RAG a nivel de arquitectura y construir un RAG funcional en Python con LangChain.

Requisitos y entorno

Los ejercicios prácticos están pensados para ejecutarse en un entorno Python 3.10+.

Instalación mínima (elige UNA opción de LLM):

Opción A: OpenAI (requiere OPENAI_API_KEY)

```
pip install -U langchain langchain-core langchain-community langchain-openai langchain-chroma chromadb tiktoken
```

Opción B: Ollama local (sin API key, requiere Ollama instalado)

```
pip install -U langchain langchain-core langchain-community langchain-ollama langchain-chroma chromadb tiktoken
```

Embeddings locales recomendados (sin token HF)

```
pip install -U langchain-huggingface sentence-transformers
```

Se permite usar cualquier corpus pequeño (p.ej., 2–5 documentos TXT/PDF), pero se recomienda preparar un directorio ./data con 3–6 ficheros (se proporcionan algunos de base sobre asuntos deportivos pero se pueden ampliar con los que se deseen).

Bloque A — Teoría (sin código)

Ejercicio A1 — RAG vs Fine-Tuning vs Prompting

Para un asistente sobre normativa deportiva interna (documentos privados que cambian cada semana): (1) ¿Cuándo elegirías RAG? (2) ¿Cuándo elegirías fine-tuning? (3) ¿Qué papel juega el prompt engineering en ambos? Da una respuesta razonada (máx. 12 líneas).

Ejercicio A2 — Evaluación de RAG

Propón un plan de evaluación con al menos 3 métricas o criterios: (i) métricas de retrieval (p.ej., Recall@k), (ii) métricas de respuesta (p.ej., exactitud/fidelidad), (iii) métricas de robustez (p.ej., sensibilidad a perturbaciones). Indica cómo obtendrías datos de evaluación (conjunto de preguntas, gold answers, etc.).



Bloque B — Práctica (Python + LangChain)

Ejercicio B1 — RAG mínimo funcional (TXT → Chroma → QA)

Construye un RAG mínimo que:

- 1) Cargue varios .txt desde ./data
- 2) Divida en chunks con RecursiveCharacterTextSplitter
- 3) Genere embeddings (HuggingFaceEmbeddings recomendado)
- 4) Indexe en Chroma (persistente)
- 5) Responda una pregunta con create_retrieval_chain
- 6) Realiza prompts que se creen resúmenes generales y por temáticas del RAG.

Requisitos:

- El LLM debe responder SOLO con base en el contexto recuperado.
- La respuesta debe incluir una sección 'Fuentes' con los metadatos (p.ej., filename) de los chunks usados.

Entrega: fragmento de código + breve análisis (10–12 líneas).

Entrega

Entrega un repositorio o un zip con:

- Notebook o script para el ejercicio práctico.
- Un breve README con instrucciones de ejecución.
- Respuestas a los ejercicios teóricos.

MATERIAL DOCENTE EN ABIERTO

Ejercicios de sistemas basados en agentes

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández.
Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons
Atribución-CompartirIgual 4.0 Internacional, disponible en
<https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y
materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la
licencia, salvo indicación expresa.

Agentes aplicados al análisis deportivo

Se plantean diferentes ejercicios basados en agentes LLM + Tools + Skills (framework práctico: LangGraph).

Objetivo: diseñar flujos agénticos controlados e integrar herramientas externas.

1) Bloque teórico (sin código)

Ejercicio T1 — Tool vs Skill vs Agent (10–12 líneas)

Define con tus palabras (sin copiar definiciones) las diferencias entre tool, skill y agente LLM. Incluye un ejemplo de cada uno en el contexto de un asistente universitario (normativa, tutorías, evaluación, etc.).

Ejercicio T2 — Diseño de control: evitar loops y alucinaciones (12–15 líneas)

Explica por qué los agentes pueden entrar en bucles o tomar acciones incorrectas. Propón mecanismos concretos de control.

Ejercicio T3 — RAG como tool dentro de un agente (8–10 líneas)

Describe cómo integrarías un sistema RAG como tool dentro de un agente. ¿Qué decide el agente? ¿Qué devuelve la tool? ¿Qué riesgos aparecen (p. ej., retrieval irrelevante) y cómo los mitigarías?

2) Bloque práctico con LangGraph

Entrega recomendada: un repositorio con scripts o notebooks por ejercicio, y un README con instrucciones. En cada ejercicio, incluye un ejemplo de ejecución y una breve discusión (5–10 líneas) de lo observado.

Ejercicio P1 — Flujo sencillo (grafo lineal con estado)

Implementa un flujo en LangGraph con 3 nodos y estado compartido:

- 1) Nodo "preprocess": normaliza la pregunta (p. ej., minúsculas, recorta espacios, detecta idioma simple).

- 2) Nodo "llm_answer": el LLM responde con un formato fijo (título + 3 bullets).

- 3) Nodo "postprocess": añade una sección final "Calidad" con dos checks: "¿hay respuesta?" y "¿es clara?".

Requisitos:

- Define un estado TypedDict con campos: question (str), normalized_question (str), answer (str).
- El grafo debe compilar y ejecutarse con `app.invoke({...})`.



- Añade logging simple para ver el paso por nodos.

Pista: este ejercicio busca que entiendas el concepto de state y cómo LangGraph estructura un pipeline, sin herramientas todavía.

Ejercicio P2 (opcional) — Agente con tool RAG (conocimiento externo)

Integra una tool tipo RAG como nodo de LangGraph (puedes reutilizar tu vector store de ejercicios de RAG). El agente debe:

- 1) Detectar si la pregunta requiere conocimiento documental.
- 2) Si sí, llamar a "rag_tool" (retrieval) y luego responder con groundedness.
- 3) Si no, responder directamente.

Requisitos:

- El estado debe incluir: question, decision, retrieved_context, sources, answer.
- La respuesta debe terminar con "Fuentes:" listando metadatos.
- Debe existir una política de abstención: si no hay contexto suficiente, decir "No puedo responder con la información disponible."

3) Entregables y criterios de evaluación

Entregables mínimos:

- Código ejecutable (scripts o notebooks) por ejercicio práctico.
- README con instalación y ejecución
- Capturas o logs de ejecuciones representativas
- Respuestas a T1–T3

Anexo — Esqueleto mínimo de LangGraph (referencia)

```
from typing import TypedDict
from langgraph.graph import StateGraph, END
```

```
class State(TypedDict):
    question: str
    answer: str
```

```
def node_a(state: State) -> State:
    # ...
    return state
```

```
graph = StateGraph(State)
graph.add_node("a", node_a)
graph.set_entry_point("a")
graph.add_edge("a", END)
```

```
app = graph.compile()  
print(app.invoke({"question": "Hola", "answer": ""}))
```

MATERIAL DOCENTE EN ABIERTO

Proyecto final transversal

Big Data Processing II

Máster Universitario en Análisis de Datos Deportivos

Curso 2026-2027

Alberto Fernández Isabel

Natalia Madrueño Sierro

Rubén Rodríguez Fernández



CC BY-SA 4.0

Atribución-CompartirIgual

Depósito institucional BURJC Digital

<https://burjcdigital.urjc.es>

Información de licencia y reutilización

Material docente en abierto de la Universidad Rey Juan Carlos.

© 2026 Alberto Fernández Isabel, Natalia Madrueño Sierro y Rubén Rodríguez Fernández. Algunos derechos reservados. Esta obra se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional, disponible en <https://creativecommons.org/licenses/by-sa/4.0/deed.es>.

La licencia se aplica al contenido original de la obra. Los logotipos, marcas, citas y materiales de terceros conservan su régimen jurídico propio y quedan excluidos de la licencia, salvo indicación expresa.



Universidad
Rey Juan Carlos



Máster Universitario en Análisis de Datos Deportivos

Práctica final

Big Data Processing II

**Sistema de análisis deportivo en tiempo real con
Kafka, Spark, LangGraph y generación
automática de informes enriquecidos mediante
RAG**

1. Contexto

Una entidad del ámbito deportivo desea construir un sistema capaz de monitorizar eventos de un partido en tiempo real y generar automáticamente un informe útil para analistas, cuerpo técnico y personal de comunicación.

El sistema debe ser capaz de:

- recibir eventos deportivos en tiempo real;
- procesarlos de forma distribuida;
- integrar datos adicionales relevantes;
- calcular métricas e indicadores de rendimiento;
- recuperar información contextual desde una base documental;
- generar un informe automático en lenguaje natural;
- coordinar el proceso final mediante un flujo de agentes implementado con LangGraph.

El objetivo de la práctica es desarrollar una solución aplicada en Python que combine tecnologías Big Data, procesamiento en streaming e inteligencia artificial generativa dentro de un caso de uso deportivo realista.

2. Objetivo general

Desarrollar un pipeline completo de analítica deportiva que procese eventos en tiempo real mediante **Kafka** y **Spark Structured Streaming**, y que produzca un informe automático final enriquecido mediante **RAG**, coordinando la recuperación y la generación mediante **LangGraph** y agentes con tools.

3. Caso de uso

Se trabajará sobre el escenario de **análisis en tiempo real de un partido de fútbol**.

Durante el partido se generan eventos como pases, tiros, goles, recuperaciones, pérdidas, faltas o sprints. Estos eventos son publicados en Kafka y consumidos por un pipeline de Spark. A partir de ellos, el sistema calcula métricas agregadas por equipo y jugador.

Una vez obtenidos los resultados analíticos, el sistema genera un informe automático del partido. Para mejorar su calidad, el informe no debe basarse únicamente en estadísticas numéricas, sino también en información recuperada desde una colección documental. Por ejemplo:

- descripciones de jugadores,
- perfiles de equipos,

- información táctica,
- resúmenes históricos,
- contexto sobre estilos de juego,
- documentación técnica sobre métricas deportivas.

La fase final del sistema deberá estar orquestada mediante un flujo de agentes en LangGraph, de forma que la recuperación de contexto y la elaboración del informe se realicen de manera estructurada y modular.

4. Requisitos de la práctica

La práctica deberá desarrollarse en Python e incluir obligatoriamente los siguientes componentes:

4.1. Ingesta de datos en tiempo real con Kafka

El sistema deberá incluir un productor que envíe eventos deportivos a un tópico de Kafka y un consumidor basado en Spark Structured Streaming que procese dichos eventos.

Cada evento deberá incluir, al menos, los siguientes campos:

- event_id
- timestamp
- match_id
- team
- player
- event_type
- zone
- minute
- value

El alumnado podrá partir de un conjunto de datos base o de un generador sintético de eventos.

4.2. Procesamiento distribuido con Spark Structured Streaming

El pipeline deberá consumir los eventos desde Kafka y realizar, al menos, las siguientes tareas:

- lectura del stream;
- definición explícita del esquema;
- validación y limpieza básica de datos;
- transformación de tipos;
- cálculo de métricas agregadas;
- generación de resultados analíticos intermedios y finales.

Se espera que el procesamiento se realice con PySpark, utilizando operaciones adecuadas para datos en streaming.

4.3. Integración de datos heterogéneos

Además del flujo principal de eventos, la solución deberá integrar al menos una fuente adicional de información estática o semiestructurada, por ejemplo:

- información de jugadores;
- alineaciones;
- zonas del campo;
- estadísticas históricas;
- catálogo de métricas;
- documentos descriptivos del contexto deportivo.

Esta integración deberá utilizarse para enriquecer el análisis o la generación del informe final.

4.4. Cálculo de métricas deportivas

El sistema deberá calcular métricas relevantes para el análisis del partido.

Como mínimo, deberán obtenerse indicadores por equipo y por jugador.

Métricas mínimas por equipo:

- número total de eventos;
- tiros;
- goles;
- faltas;
- recuperaciones;
- intensidad por intervalos temporales.
- Métricas mínimas por jugador
- participación total;
- eventos ofensivos;
- eventos defensivos;
- contribución acumulada al juego.

Se valorará que las métricas tengan sentido deportivo y estén correctamente justificadas.

4.5. Persistencia de resultados

Los resultados del pipeline deberán almacenarse de forma estructurada.

Como mínimo, deberán persistirse:

- los datos procesados o depurados;

- las métricas agregadas finales;
- el informe generado.

Se recomienda usar formatos como Parquet, CSV o una base de datos ligera, siempre que la elección esté justificada.

4.6. Optimización y escalabilidad

La práctica deberá incluir un apartado de optimización técnica del pipeline. El alumnado deberá aplicar y justificar al menos tres medidas relacionadas con eficiencia o escalabilidad, por ejemplo:

- filtrado temprano;
- selección de columnas necesarias;
- particionado;
- uso razonado de caché o persistencia;
- elección adecuada del formato de salida;
- control del tamaño de lotes o microbatches;
- simplificación de transformaciones.

No se exige una optimización avanzada de producción, pero sí una reflexión técnica coherente.

4.7. Generación automática de informe

A partir de las métricas calculadas, el sistema deberá generar un informe automático del partido en lenguaje natural.

El informe deberá incluir, como mínimo:

- resumen general del partido;
- equipo o jugador destacado;
- momento o tramo de mayor intensidad;
- interpretación básica del rendimiento observado;
- conclusión final orientada a un stakeholder no técnico.

4.8. Enriquecimiento mediante RAG

La generación del informe deberá apoyarse en un módulo de Retrieval-Augmented Generation (RAG).

Este módulo tendrá como objetivo recuperar información relevante desde una pequeña base documental y utilizarla para enriquecer el informe final.

La colección documental podrá contener, por ejemplo:

- descripciones de jugadores;
- estilos tácticos de equipos;

- resúmenes de encuentros anteriores;
- notas técnicas sobre indicadores de rendimiento;
- explicaciones de conceptos deportivos.

El alumnado deberá implementar un flujo básico de RAG que incluya:

- preparación o indexación de documentos;
- recuperación de contexto relevante ante una consulta;
- uso del contexto recuperado en el prompt final del modelo generativo.

El objetivo no es construir un sistema conversacional complejo, sino demostrar cómo una arquitectura RAG puede aportar contexto útil y verificable al informe generado.

Importante: el informe final deberá distinguir con claridad entre:

- información derivada de los datos del partido procesados en tiempo real;
- información contextual aportada por los documentos recuperados.

4.9. Orquestación con LangGraph y agentes con tools

La fase final de análisis y generación del informe deberá implementarse mediante LangGraph.

No será suficiente con realizar una única llamada directa a un modelo generativo. El alumnado deberá construir un flujo basado en nodos o agentes que coordine, al menos, las siguientes funciones:

- consulta de métricas generadas por el pipeline;
- recuperación de documentos relevantes del módulo RAG;
- elaboración del informe final a partir de la información disponible.

Para ello, deberán emplearse agentes con tools. Como mínimo, la solución deberá incluir:

- Tool 1. Consulta de métricas del partido
 - o Una herramienta que permita acceder a las métricas ya calculadas por Spark y almacenadas por el sistema.
 - o Ejemplos de consultas:
 - equipo con mayor número de recuperaciones;
 - jugador con mayor participación;
 - tramo temporal con más intensidad;
 - número total de tiros o faltas.
- Tool 2. Recuperación documental
 - o Una herramienta conectada al índice o base documental del módulo RAG para recuperar contexto relevante.
 - o Ejemplos:
 - descripción táctica de un equipo;

- perfil de un jugador;
- explicación de una métrica concreta;
- contexto histórico relacionado con el partido analizado.

También podrá plantearse una arquitectura con dos agentes especializados y un nodo final de consolidación.

Lo importante es que:

- exista una orquestación real en LangGraph;
- los agentes utilicen tools de forma explícita;
- el uso de agentes aporte estructura y trazabilidad al proceso;
- el resultado final esté conectado de forma coherente con el pipeline de Big Data.

No se exige un sistema multiagente excesivamente complejo, pero sí una implementación clara, modular y funcional.

5. Salida final del sistema

La práctica deberá producir una salida final comprensible y bien presentada. Esta salida podrá adoptar una de las siguientes formas:

- notebook con tablas, gráficos e informe final;
- aplicación sencilla en Streamlit;
- informe HTML o PDF generado automáticamente.

La salida deberá mostrar de forma clara:

- métricas obtenidas;
- elementos más relevantes del análisis;
- contexto recuperado desde la base documental;
- informe narrativo final generado mediante el flujo en LangGraph.

6. Tecnologías mínimas

La práctica deberá desarrollarse utilizando, como mínimo:

- Python
- Apache Kafka
- PySpark / Spark Structured Streaming
- LangGraph

Además, será necesario incorporar alguna solución para:

- indexación y recuperación documental;
- embeddings o búsqueda semántica;
- generación de texto mediante un modelo generativo.

El alumnado podrá elegir la tecnología concreta, siempre que quede correctamente explicada.

7. Entregables

7.1. Código fuente

El alumnado deberá entregar el código completo, organizado de forma clara.

La estructura recomendada es la siguiente:

```
final_project/
├── data/
│   ├── static/
│   └── docs/
├── output/
│   ├── processed/
│   ├── aggregates/
│   └── reports/
├── src/
│   ├── kafka_producer.py
│   ├── streaming_pipeline.py
│   ├── metrics.py
│   ├── rag_pipeline.py
│   ├── tools.py
│   ├── graph_workflow.py
│   ├── report_generator.py
│   └── app.py
├── requirements.txt
├── README.md
└── memoria.pdf
```

7.2. Memoria técnica

La memoria deberá describir de forma clara la solución implementada y deberá incluir, al menos:

- objetivo del proyecto;
- arquitectura general;
- descripción de los datos;

- uso de Kafka;
- procesamiento con Spark;
- métricas calculadas;
- diseño del módulo RAG;
- diseño del flujo en LangGraph;
- tools implementadas y función de cada una;
- generación del informe;
- decisiones de optimización;
- limitaciones del sistema.

7.3. Presentación oral

La práctica se defenderá oralmente en la fecha indicada oficialmente en la convocatoria de exámenes.

La presentación deberá mostrar:

- el problema abordado;
- la arquitectura desarrollada;
- una demostración del flujo completo;
- el papel de Kafka, Spark, RAG y LangGraph;
- los principales resultados;
- las decisiones técnicas adoptadas.

8. Criterios generales de valoración

En la evaluación se tendrá en cuenta:

- el correcto funcionamiento del pipeline completo;
- la integración adecuada de Kafka y Spark;
- la calidad del análisis deportivo realizado;
- la coherencia del módulo RAG dentro de la solución;
- la correcta utilización de LangGraph y agentes con tools;
- la calidad del informe automático generado;
- la justificación técnica de las decisiones adoptadas;

- la claridad de la memoria y de la presentación oral.

9. Consideraciones importantes

- La práctica debe funcionar de extremo a extremo.
- No se valorará positivamente incluir tecnologías de forma superficial.
- Kafka debe utilizarse como mecanismo real de ingesta en tiempo real.
- El RAG debe aportar contexto útil al informe final.
- LangGraph debe emplearse para coordinar de forma explícita la fase final del sistema.
- Los agentes deberán utilizar tools reales conectadas con los datos y documentos del proyecto.
- La solución debe estar orientada a un caso de uso deportivo realista.
- El código deberá ser legible, modular y reproducible.

10. Integridad académica

Todo el trabajo entregado debe ser original o estar correctamente referenciado. El uso de asistentes de IA deberá limitarse a tareas de apoyo y nunca podrá sustituir la comprensión técnica del proyecto ni la autoría efectiva del código y de la memoria.

MATERIAL DOCENTE EN ABIERTO